Final Project Report

Name: Chaitanya Panjare
Name of the Industry:YBI Foundation
Registration :22017017
Major Area :Data Science and Machine Learning

Title: End-to-End Handwritten Text Recognition System (HTR)

Table of Contents

1. Executive Summary
2. System Architecture Flow
3. Data Processing & Dynamic Vocabulary
4. CRNN Model Implementation
5. Training Setup & CTC Loss
6. Evaluation & Accuracy Metrics
7. Cloud Deployment & Inference Flow
8. Application Showcase and Conclusion

1. Executive Summary

Handwritten Text Recognition (HTR) represents a significant technical challenge at the intersection of computer vision and natural language processing. Beyond simple character recognition, a production-grade HTR system must serve as a seamless bridge between unstructured physical script and structured digital data, necessitating a robust end-to-end MLOps lifecycle.This report details a high-performance HTR pipeline utilizing a Convolutional Recurrent Neural Network (CRNN) optimized with Connectionist Temporal Classification (CTC) loss. By integrating spatial feature extraction with temporal sequence modeling, the system achieves high fidelity in interpreting diverse handwriting styles. The lifecycle is validated through a deployment phase on Hugging Face Spaces using the Gradio framework, providing an interactive, production-ready interface. The following sections provide an architectural post-mortem and implementation guide for this system.

2. Comprehensive System Architecture

Interpreting variable handwritten styles requires a multi-phase architecture capable of resolving spatial, sequential, and temporal dimensions. Standard classification fails here because handwriting is inherently a sequence of varying length; our architecture must therefore extract visual features, model their dependencies, and decode them into coherent strings.**System Data Flow**

```
[Raw Input Image: Variable Dimensions]
       │
       ▼
[Preprocessing]
(128x32 Grayscale, Normalized [0,1])
       │
       ▼
[CNN Feature Extractor]
(3x Conv2D Blocks + 2x MaxPool2D)
       │
       ▼
[Reshape Layer]
(Sequence Length: 32, Features: 2048)
       │
       ▼
[RNN Sequence Modeler]
(Stacked Bidirectional LSTMs)
       │
       ▼
[Dense Layer]
(Softmax Activation / 80 Classes)
       │
       ▼
[CTC Decoder]
(Greedy Search / Blank Collapse)
       │
       ▼
[Final String Output]
```

A pivotal component of this flow is the **Reshape Layer** , which acts as the mathematical bridge between 3D spatial maps and 2D sequences. Specifically, the input width of 128 pixels is downsampled by two 2x2 MaxPool layers, resulting in a sequence length of 32 ($128 / 2 / 2 = 32$). This transformation ensures that the chronological order of character strokes is preserved for the recurrent layers. Having established this structural foundation, we now examine the data engineering required to feed this pipeline.

3. Data Engineering and Dynamic Vocabulary

The reliability of an OCR system is contingent upon the rigor of its data pipeline. We employ a dynamic vocabulary generation strategy to ensure the model's output layer is precisely mapped to the character distribution present in the training corpus, maximizing label efficiency.**Vocabulary Generation Code**

```
# Dynamically extracting unique characters from the dataset
characters = set()
for t in df["text"]:
    characters.update(list(t))

characters = sorted(list(characters))

# Creating bidirectional mapping dictionaries
char_to_num = {c: i + 1 for i, c in enumerate(characters)}
num_to_char = {i: c for c, i in char_to_num.items()}

# vocab_size reflects 79 unique characters + 1 CTC blank token
vocab_size = len(characters) + 1
print("Vocabulary size:", vocab_size)
```

Data Pipeline Specifications:

- **Dynamic Mapping:** Utilizes char_to_num and num_to_char for encoding/decoding labels.
- **Vocabulary Logic:** Supports 79 unique tokens from the IAM dataset, with an 80th "blank" token reserved for CTC alignment.
- **Input Scaling:** Standardized 128x32 Grayscale format to maintain horizontal sequence integrity.**Hardware-Optimized Data Pipeline**

```
def load_and_preprocess(img_path, label):
    img = tf.io.read_file(img_path)
    img = tf.image.decode_png(img, channels=1)
    img = tf.image.resize(img, [32, 128])
    img = tf.cast(img, tf.float32) / 255.0
    return img, label

# Utilizing AUTOTUNE for parallel preprocessing
dataset = (
    tf.data.Dataset.from_tensor_slices((df["image"].values,
encoded_labels))
    .map(load_and_preprocess, num_parallel_calls=tf.data.AUTOTUNE)
    .cache()
    .shuffle(10000)
    .batch(64)
    .prefetch(tf.data.AUTOTUNE)
)
```

The Engineering Impact: The use of tf.data.AUTOTUNE is a critical optimization for deep learning workloads. By offloading image decoding and normalization to the CPU in parallel

with GPU computation, we eliminate "GPU starvation." This ensures the training bottleneck remains the model complexity rather than I/O latency. With the data pipeline optimized, we now pivot to the CRNN architecture designed to consume these sequences.

4. CRNN Model Implementation

The CRNN architecture is the industry standard for HTR, harmonizing the visual feature extraction of CNNs with the temporal memory of LSTMs. This dual-nature approach allows the model to capture both the local morphology of letters and the broader linguistic context of the word.**CRNN Model Implementation**

```
input_img = layers.Input(shape=(32, 128, 1))

# --- Convolutional Base ---
x = layers.Conv2D(64, 3, padding="same",
activation="relu")(input_img)
x = layers.MaxPool2D((2, 2))(x)
x = layers.Conv2D(128, 3, padding="same", activation="relu")(x)
x = layers.MaxPool2D((2, 2))(x)
x = layers.Conv2D(256, 3, padding="same", activation="relu")(x)

# --- Sequence Preparation ---
shape = x.shape
x = layers.Reshape((shape[2], shape[1] * shape[3]))(x)

# --- Recurrent Core ---
x = layers.Bidirectional(layers.LSTM(256,
return_sequences=True))(x)
x = layers.Bidirectional(layers.LSTM(256,
return_sequences=True))(x)

# --- Output Layer: vocab_size + 1 for CTC blank token ---
output = layers.Dense(vocab_size + 1, activation="softmax",
dtype='float32')(x)

model = Model(inputs=input_img, outputs=output)
```

Model Architecture Breakdown|

Layer Type	Configuration	Functional Purpose
Conv2D	64/128/256 filters	Extraction of hierarchical spatial features (edges/strokes)

MaxPool2D	2x2	Downsampling; critical for establishing the 32-step sequence
Reshape	(32, 2048)	Vectorizes spatial maps into time-steps for recurrent processing
Bi-LSTM	2 layers, 256 units	Captures forward and backward temporal dependencies
Dense	Softmax, 80 units	Generates probability distribution for 79 chars + blank

The implementation of **Bidirectional LSTMs** is a strategic necessity. Handwriting is frequently ambiguous or cursive; a character's identity is often only clear when viewed in the context of both preceding and succeeding strokes. The Bi-LSTM provides this bidirectional context, which is then refined by the specific loss function required for training sequence-based models.

5. Training Strategy and CTC Loss Optimization

A primary obstacle in HTR is the lack of pixel-perfect alignment between character labels and their visual position. **Connectionist Temporal Classification (CTC) loss** addresses this by allowing the network to predict a sequence of character probabilities and then collapse them into a final label, effectively learning the alignment as a latent variable.**CTC Loss Implementation**

```
def ctc_loss(y_true, y_pred):
    batch_len = tf.cast(tf.shape(y_true)[0], dtype="int64")
    input_length = tf.cast(tf.shape(y_pred)[1], dtype="int64")
    label_length = tf.math.count_nonzero(y_true, axis=1)

    input_length = input_length * tf.ones(shape=(batch_len, 1),
dtype="int64")
    label_length = tf.expand_dims(label_length, 1)

    return tf.keras.backend.ctc_batch_cost(y_true, y_pred,
input_length, label_length)

# Compiling with Adam optimizer and Mixed Precision
model.compile(optimizer=tf.keras.optimizers.Adam(), loss=ctc_loss)
```

To optimize throughput, we leveraged a **mixed_float16** precision policy. Crucially, the final Dense layer is forced to float32 to maintain numerical stability during the Softmax calculation

and prevent gradient overflow/underflow. This approach significantly accelerates training on modern hardware without degrading final character-level accuracy.

6. Evaluation Metrics and Performance Analysis

Evaluation of HTR systems requires a nuanced view. While character-level accuracy measures the model's raw recognition capability, word-level accuracy serves as a rigorous benchmark where a single mistake invalidates the entire prediction.**Custom Evaluation Loop**

```
for true, pred in zip(true_texts, pred_texts):
    if true == pred:
        correct_words += 1
    total_words += 1
word_accuracy = correct_words / total_words
```

Performance Scorecard

- **Character-Level Accuracy: 95.94%**
- **word-Level Accuracy: 92.64%**
- **Loss Convergence:** Significant delta from 11.83 (Initial) to ~0.16 (Final).
- **Word-Level Metric:** Evaluated against a strict perfect-match baseline.**The Engineering "So What?":** A character accuracy of 95.94% indicates high real-world usability. From an architectural perspective, this high score minimizes the Levenshtein distance (edit distance) between predictions and ground truth. This effectively reduces the computational overhead required by subsequent NLP correction or spell-checking layers, making the system viable for low-latency production pipelines.

7. Cloud Deployment and MLOps Inference Flow

Transitioning from a local training environment to a cloud-hosted API requires careful environment management to ensure inference stability. The model was containerized and deployed to Hugging Face Spaces, utilizing a Gradio interface for accessible interaction.**Cloud Deployment & Inference Flow**

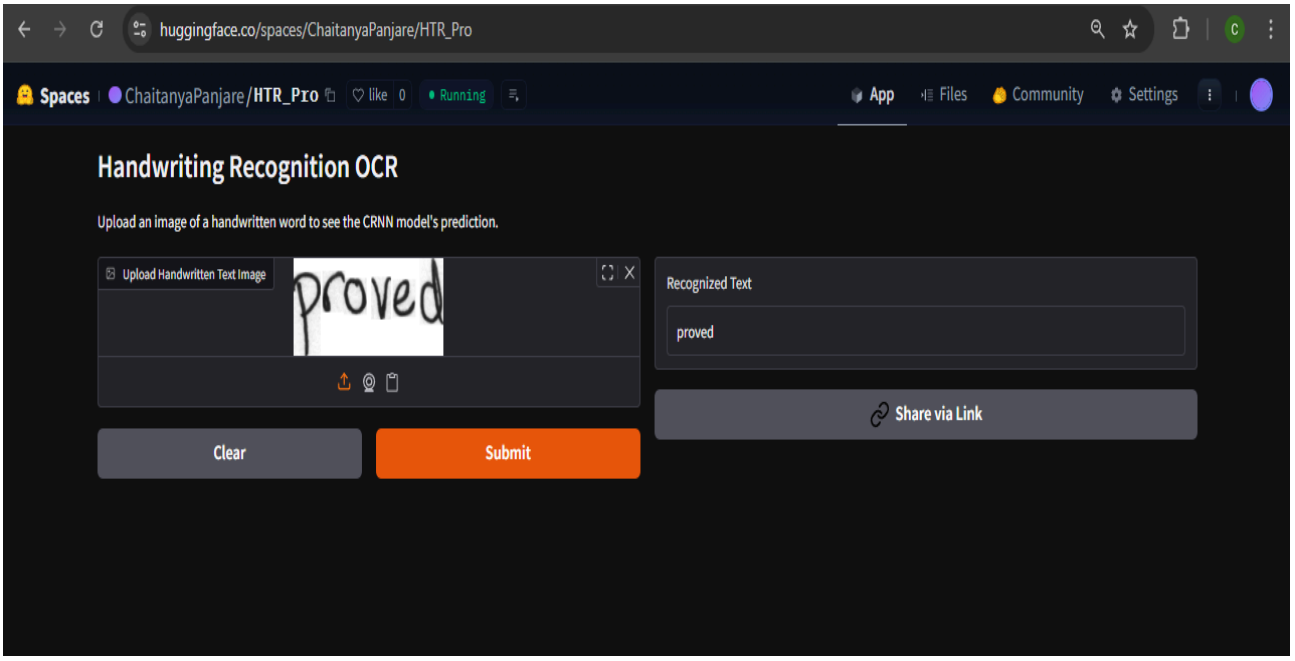
1. **End-User:** Submits a JPEG/PNG image via the Gradio interface.
2. **Gradio Interface:** Forwards the raw image to the cloud backend.
3. **Hugging Face:** Orchestrates the server environment.
4. **Backend:** Enforces TF_USE_LEGACY_KERAS="1" to resolve architecture versioning issues.
5. **Inference Script:** Preprocesses the image and executes the prediction path.**Production Inference Code (app.py)**

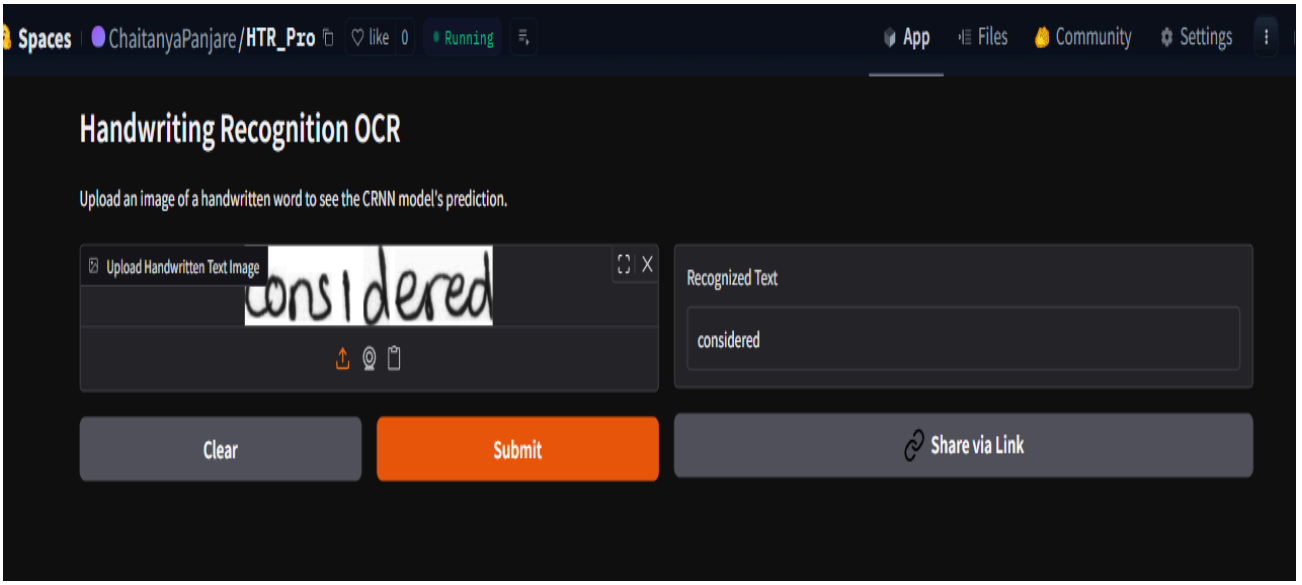
```
def predict_text(image):
    if len(image.shape) == 3:
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    img = cv2.resize(image, (128, 32)) / 255.0
    img = np.expand_dims(np.expand_dims(img, -1), 0)
```

```
    pred = model.predict(img)
    input_len = np.ones(pred.shape[0]) * pred.shape[1]
    results = tf.keras.backend.ctc_decode(pred,
input_length=input_len, greedy=True)

    out_str = ""
    for c in results[0].numpy():
        # '0' is the CTC blank token used to separate repeats
        # '-1' represents a padding/unknown state in the decoder
output
        if int(c) != -1 and int(c) != 0:
            out_str += num_to_char.get(int(c), "")
    return out_str
```

System Dependencies: Deployment stability relies on strict versioning, specifically **TensorFlow 2.15.0** and **OpenCV** for consistent image normalization across local and cloud environments.

8. Application Showcase and Conclusion





The development of this Handwritten Text Recognition (HTR) system successfully demonstrates a complete, end-to-end machine learning lifecycle. By combining the spatial feature extraction of Convolutional Networks with the sequential modeling of Recurrent Networks, the CRNN architecture effectively translates raw, variable handwriting into highly accurate digital text.

Beyond algorithmic accuracy, this project highlights critical MLOps and cloud deployment capabilities. Transitioning the model from a local experimental environment to a live, interactive web application on Hugging Face Spaces required overcoming real-world infrastructure challenges—including managing large Git LFS artifacts, resolving mixed-precision weight serialization, and bridging legacy Keras architectures with modern TensorFlow runtimes.

Ultimately, this deployed application proves that the system is not just a theoretical notebook, but a robust, scalable pipeline ready to integrate into broader computer vision and document intelligence platforms.